

An aerial photograph of a river winding through a snowy landscape. The river is dark and narrow, contrasting with the bright white snow. The snow is covered with small, dark patches of vegetation and some tracks. The river flows from the top right towards the bottom left, curving along the way.

Aula 7

Pré-treinamento e BERT

Tópicos Avançados em Inteligência Artificial · UFABC

Roteiro da aula

Parte 1 — Representações como Conhecimento Redes neurais como aprendedores de representação

Parte 2 — Self-Supervised Learning Criando labels a partir dos próprios dados

Parte 3 — BERT Arquitetura, pré-treinamento e fine-tuning

Parte 4 — HuggingFace Hub Ecossistema de modelos pré-treinados

Parte 5 — Transformers Além do NLP ViT, TabTransformer e multimodalidade

O Transformer transforma embeddings

"Jacob **Station** lives nearby the train **station** close to the radio **station**"



barras ilustrativas — valores reais visualizados no notebook

Parte 1 — Representações como Conhecimento

Redes neurais aprendem representações

A saída de cada camada é uma representação

A entrada original é transformada progressivamente a cada camada oculta.

Em vez de "resposta", cada camada produz uma nova visão dos dados.

Hierarquia visual em CNNs

Camadas iniciais detectam **linhas e bordas**

Camadas intermediárias detectam **formas e texturas**

Camadas profundas detectam **objetos complexos**

Camada final detecta **categorias semânticas**

Intuição: Uma rede treinada para classificar 1000 categorias ImageNet não decorou pixels — ela aprendeu uma hierarquia de conceitos visuais. As representações intermediárias codificam conhecimento geral sobre o mundo visual.

Transfer Learning: aproveitando representações

Cenário clássico

1. Treinar ResNet em ImageNet (1.2M imagens, 1000 classes)
2. Remover a camada de saída
3. Usar o restante como **encoder fixo**
4. Adicionar nova cabeça para tarefa alvo

Por que funciona?

Com apenas 100 exemplos de bolsas vs sapatos, o encoder já capturou:

- formas e contornos
- texturas e materiais
- partes de objetos

Fluxo do Transfer Learning

```
ResNet (pré-treinado ImageNet)
    ↓
[remover última camada]
    ↓
Encoder de representações
    ↓
Dense → softmax ← treinar com 100 exemplos
```

Representações de camadas profundas generalizam para tarefas relacionadas sem precisar de milhões de exemplos na tarefa alvo.

Para NLP: precisamos de um encoder de texto

O que queremos

Um encoder que produza **representações ricas** de texto: capture semântica, sintaxe, contexto, relações entre palavras.

O problema das labels

Para treinar com supervisão, precisamos de labels anotadas por humanos.

Anotação é cara e lenta — e muitas tarefas têm poucos exemplos disponíveis.

A boa notícia

Texto puro está disponível em quantidade praticamente ilimitada:

- Wikipedia (bilhões de palavras)
- Livros digitais (BooksCorpus, Project Gutenberg)
- Páginas da internet (Common Crawl)

Parte 2 — Self-Supervised Learning

O problema das labels custosas

Aprendizado supervisionado tradicional

Requer pares (entrada, label) anotados manualmente.

Classificar sentimento: humano lê frase e marca positivo/negativo.

NER: humano identifica cada entidade no texto.

Custo: horas de trabalho especializado por milhares de exemplos.

O que temos em abundância

Bilhões de frases não rotuladas: livros, artigos, fóruns, código-fonte.

Nenhuma label humana — mas toda a estrutura da linguagem está presente.

Pergunta: podemos aprender representações úteis sem labels?

Self-Supervised Learning: criar labels artificiais automaticamente a partir da estrutura dos próprios dados. Usar parte da entrada para prever outra parte da mesma entrada.

Mascaramento: a ideia central

Procedimento de mascaramento

1. Pegar texto original
2. Substituir tokens aleatórios por MASK
3. Treinar modelo para prever os tokens mascarados
4. Labels = tokens originais (extraídos automaticamente)

Nenhuma anotação humana necessária.

Criação de pares de treinamento

Entrada mascarada → Label = token original

Geração totalmente automática a partir de qualquer corpus de texto.

Exemplo concreto:

Original: "The mission of the MIT Sloan School of Management is to develop principled, innovative leaders..."

Mascarado: "The MASK of the MIT Sloan School of MASK is to develop principled, innovative MASK..."

Labels: "mission", "Management", "leaders"

Exemplo: preenchendo lacunas

Texto de entrada (com máscaras):

"The MASK of the MIT Sloan School of MASK is to develop principled, innovative MASK who improve the world and to generate ideas that advance management practice."

MASK 1

Para prever "mission", o modelo precisa entender que "The ___ of the MIT Sloan School" pede um substantivo de propósito institucional.

Requer **semântica** e **estrutura sintática**.

MASK 2

"MIT Sloan School of ___" — o modelo precisa conhecer que escolas têm nomes completos e que "Management" completa esse nome.

Requer **conhecimento de mundo**.

MASK 3

"develop principled, innovative ___" — o modelo infere que pessoas (leaders/thinkers) completam a frase.

Requer **relações semânticas**.

Por que o mascaramento funciona?

Para preencher lacunas, o modelo precisa:

- Entender o significado das palavras ao redor
- Capturar dependências de longa distância
- Modelar coerência semântica e gramatical
- Integrar contexto bidirecional (antes e depois)

Resultado

As representações internas aprendidas durante o mascaramento codificam automaticamente todo esse conhecimento linguístico — sem nenhuma label humana.

Como tarefa de sequência:

Entrada mascarada



Transformer Encoder



Embedding por token



Dense(ReLU)



Softmax → token previsto

É um problema de rotulagem de sequência: cada token mascarado tem um label (o token original). O modelo computa a distribuição sobre o vocabulário para cada MASK.

Parte 3 — BERT

BERT: visão geral

BERT

Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 — Google AI

Arquitetura

Transformer Encoder pré-treinado com:

- Masked Language Modeling (MLM)
- Next Sentence Prediction (NSP)

Corpus: Wikipedia + BooksCorpus (~3.3B palavras)

Variantes originais

BERT-base: 12 camadas, 768 dim, 12 heads — 110M parâmetros

BERT-large: 24 camadas, 1024 dim, 16 heads — 340M parâmetros

Pré-treinamento: semanas em TPUs. Fine-tuning: minutos/horas em GPU.

Ideia central: pré-treinar uma vez em texto massivo, depois adaptar rapidamente para qualquer tarefa com poucos exemplos rotulados.

BERT é bidirecional — o que isso significa?

Modelos unidirecionais (ex: GPT)

Ao processar a palavra "bank" na posição 4, o modelo só enxerga os tokens anteriores:

She went to the **bank** to deposit money

O contexto à **direita** está bloqueado — necessário para gerar texto, mas limitante para compreensão.

BERT: atenção bidirecional total

Ao codificar "bank", o modelo vê **todos** os tokens:

She went to the **bank** to deposit money

Por que o MLM permite bidirecionalidade?

No next-word-prediction (GPT), ver o futuro seria "colar na resposta" — o modelo simplesmente copiaria o próximo token.

No MLM (BERT), o token alvo está **ausente** da entrada ([MASK]).

Usar o contexto dos dois lados para prever o token faltante é legítimo — é exatamente o que queremos.

Máscara de atenção: a diferença visual

GPT (causal)



BERT (bidirecional)



Arquitetura BERT: encoder stack

[CLS] tok₁ tok₂ ... tok_n [SEP]

Embedding de entrada

Token emb. + Segment emb. + Position emb. (learnable)
→ soma elemento a elemento → dim 768

Bloco Encoder × 12 (base) / × 24 (large)

- ① Multi-Head Self-Attention 12 cabeças, d_k = 64
- ② Add & LayerNorm residual + norm
- ③ FFN: Linear → GeLU → Linear 768 → 3072 → 768
- ④ Add & LayerNorm residual + norm

Bloco Encoder × 12 ...

Bloco Encoder × 12 ...

h_{CLS} h₁ h₂ ... h_n (dim 768 cada)

BERT-base vs BERT-large

Config	base	large
Camadas (L)	12	24
Dimensão (H)	768	1024
Cabeças (A)	12	16
FFN dim	3072	4096
Parâmetros	110M	340M

Três embeddings somados na entrada

Token: lookup — 30k tokens (WordPiece)

Segment: A ou B — pares de sentenças

Position: learnable (≠ senos/cossenos do Transformer original)

Por que GeLU no FFN? Gradientes mais suaves que ReLU;
empiricamente converge melhor em LLMs.

Fine-tuning BERT: estratégias e hiperparâmetros

Estratégia 1 — Encoder congelado

Congela o BERT, treina apenas a cabeça. Rápido, funciona com dezenas de exemplos; não adapta representações.

Estratégia 2 — Full fine-tuning

Atualiza todos os pesos com lr baixo ($\sim 2e-5$). Melhor desempenho, mas lr alto causa **catastrophic forgetting**.

Estratégia 3 — Layer-wise LR decay

Camadas profundas = lr menor; camadas de saída = lr maior. Preserva representações gerais, adapta o topo.

Hiperparâmetros recomendados (Devlin et al., 2018)

lr:	2e-5, 3e-5, 5e-5
batch size:	16, 32
epochs:	2-4
warmup:	10%
weight decay:	0.01

Quando usar cada estratégia?

Dados disponíveis	Estratégia
< 100 exemplos	Encoder congelado
100-5k	Full fine-tuning
> 5k	Full + layer-wise decay

O token CLS: representação global

Token especial CLS

Adicionado no início de cada sequência de entrada.

Ao longo do pré-treinamento, o embedding de CLS aprende a agregar informação de toda a sequência via self-attention.

Ao final, representa a sentença como um todo — contexto global.

Estrutura de entrada BERT

CLS] token1 token2 ... tokenN [SEP

↓	↓	↓		↓
h_{CLS}	h_1	h_2	...	h_N

h_{CLS} : embedding global da sentença

h_i : embedding contextualizado do token i

Por que CLS funciona como representação global?

Sentence-BERT: além do CLS

O embedding de [CLS] é ótimo para classificação, mas **não é ideal para similaridade semântica**.

Limitação do CLS para similaridade

Comparar N sentenças exige N^2 inferências. Além disso, distância coseno entre [CLS] de sequências independentes **não correlaciona bem** com similaridade semântica humana.

Solução: Sentence-BERT (Reimers & Gurevych, 2019)

BERT treinado com **rede siamesa** em pares de similaridade. Usa **mean pooling** (média dos tokens) em vez de [CLS].

Resultado: N inferências, não N^2 . Embeddings comparáveis por coseno.

Arquitetura siamesa:

```
sentença A → BERT → mean pool → u ┐  
                                     │  
sentença B → BERT → mean pool → v ┘ concat(u, v, |u-v|) → similaridade  
      (pesos compartilhados – treinado com NLI)
```

Fine-tuning para classificação de sequência

Tarefa: análise de sentimento

Dada uma frase, classificar como positivo ou negativo.

Arquitetura de fine-tuning

Entrada: [CLS] I loved this movie [SEP]

↓

BERT Encoder (pré-treinado)

↓

Embedding de [CLS]: h_{CLS}

↓

Dense(ReLU)

↓

sigmoid / softmax

↓

positivo / negativo

Estratégias de fine-tuning

Treinar apenas a cabeça: BERT congelado, treinar somente Dense + softmax. Rápido, bom com poucos exemplos.

Full fine-tuning: atualizar todos os pesos do BERT junto com a cabeça. Melhor desempenho, precisa de mais dados e tempo.

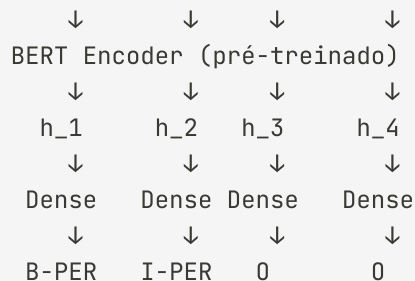
Fine-tuning para rotulagem de sequência

Tarefas: NER, slot filling, POS tagging

Cada token recebe uma classe (ex: pessoa, organização, local, outro).

Arquitetura de fine-tuning

CLS] Barack Obama foi presidente [SEP



Flexibilidade do BERT

A mesma arquitetura pré-treinada serve para ambas as tarefas. O que muda é apenas a cabeça adicionada no fine-tuning:

- Classificação: usar h_{CLS}
- Rotulagem: usar h_i de cada token

Variações do BERT

RoBERTa (Liu et al., 2019 — Facebook)

Melhorias sobre BERT:

- Treinado por mais tempo
- Batch maior (8k amostras)
- Remove NSP
- Mais dados de treinamento

Melhor desempenho em benchmarks de NLU.

DistilBERT (Sanh et al., 2019 — HuggingFace)

Destilação do conhecimento de BERT:

- 40% menor
- 60% mais rápido
- 97% do desempenho

Ideal para produção com restrição de recursos.

BERT Multilingual (Google, 2018)

- 104 idiomas
- Um único modelo
- Treinado em Wikipedia multilíngue

Funciona para português, espanhol, chinês, árabe, etc. — sem fine-tuning por língua.

Sentence Transformers (sbert.net): variantes de BERT otimizadas para gerar embeddings de sentenças semanticamente similares. Usam treinamento contrastivo com pares de sentenças. Ideais para busca semântica, deduplicação e clustering.

Parte 4 — HuggingFace Hub

HuggingFace: o ecossistema de modelos

O Hub

500k+ modelos pré-treinados, datasets e spaces de demonstração — NLP, visão, áudio, multimodal, dezenas de idiomas. Qualquer pesquisador ou empresa pode publicar e baixar.

Workflow típico

1. Escolher modelo no Hub para a tarefa
2. `AutoTokenizer` / `AutoModel` para carregar
3. Adicionar cabeça de tarefa (classificação, NER...)
4. Fine-tuning com poucos exemplos rotulados

Conceito chave

Use BERT (ou variante) como encoder pré-treinado e treine apenas a cabeça com seus exemplos. Não precisa de milhões de dados — o BERT já sabe sobre linguagem.

Código: HuggingFace Transformers

```
from transformers import pipeline

# Análise de sentimento com BERT
classifier = pipeline("sentiment-analysis")
result = classifier("I loved the movie!")
# [{'label': 'POSITIVE', 'score': 0.9998}]

# Fill-mask
fill = pipeline("fill-mask", model="bert-base-uncased")
fill("The capital of France is [MASK].")
# [{'token_str': 'paris', 'score': 0.9975}]

# NER
ner = pipeline("ner", aggregation_strategy="simple")
ner("Hugging Face is based in New York")
```

pipeline() carrega automaticamente modelo + tokenizer + pós-processamento para a tarefa escolhida. Basta passar o nome da tarefa.

AutoTokenizer / AutoModelForSequenceClassification permitem carregar qualquer modelo do Hub e adicionar cabeças personalizadas para fine-tuning completo.

Parte 5 — Transformers Além do NLP

Vision Transformer (ViT)

Ideia: tratar imagem como sequência de patches

1. Dividir imagem em patches de tamanho fixo (ex: 16×16 pixels)
2. Achatar cada patch em um vetor
3. Projeção linear $\rightarrow$ embedding de patch
4. Adicionar positional encoding
5. Alimentar Transformer Encoder
6. Usar CLS para classificação

Resultado

Dosovitskiy et al., 2020 (Google Brain):

Com escala suficiente de dados de pré-treinamento, ViT é competitivo ou superior a CNNs em classificação de imagens.

Fluxo do ViT:

```
Imagem 224x224
  ↓
Dividir em patches 16x16 → 196 patches
  ↓
Achatar + Linear Projection → 196 embeddings
  ↓
[CLS] + positional encoding
  ↓
Transformer Encoder (igual ao de NLP)
  ↓
h_CLS → Dense → classe da imagem
```

TabTransformer: dados tabulares

Problema com dados tabulares

Variáveis categóricas têm interações complexas que modelos lineares não capturam bem.

Árvores capturam interações mas não generalizam com poucos dados.

Arquitetura TabTransformer (Huang et al., 2020)

Variáveis categóricas



Embeddings por coluna



Transformer Encoder

(captura interações entre colunas)



Variáveis numéricas → concatenar



Dense layers → predição

Vantagem: o Transformer captura automaticamente quais combinações de colunas são relevantes para a predição — algo que engenharia de features manual tentaria replicar. Especialmente útil quando variáveis categóricas interagem de forma não óbvia.

Embeddings: a linguagem comum

Texto

Tokenizar → embedding por token

O mesmo Transformer Encoder processa a sequência de embeddings.

Imagem

Dividir em patches → projeção linear

Patches viram tokens — mesmo Transformer.

Tabela / Áudio / Grafo

Qualquer dado pode ser tokenizado e projetado em embeddings de mesma dimensão.

Poder unificador dos embeddings

O Transformer Encoder é agnóstico ao tipo de dado — só precisa de uma sequência de vetores de dimensão fixa. O que muda entre domínios é apenas o módulo de tokenização/projeção.

Isso habilita modelos **multimodais**: combinar texto + imagem + tabela na mesma arquitetura, misturando embeddings de diferentes domínios na mesma sequência. CLIP, Flamingo, GPT-4V são exemplos desse paradigma.

Resumo: o que aprendemos hoje

Representações como conhecimento

Redes neurais aprendem hierarquias de representação. Transfer learning reutiliza esse conhecimento em novas tarefas.

Self-Supervised Learning

Labels artificiais criadas a partir dos dados. Mascaramento força o modelo a aprender linguagem sem supervisão humana.

BERT e fine-tuning

Encoder pré-treinado em texto massivo. `[CLS]` para classificação; `h_i` por token para rotulagem.

HuggingFace

500k+ modelos prontos. Pipeline em 3 linhas: escolher modelo → tokenizer → cabeça → fine-tune.

Transformers além do NLP

ViT (imagens como patches), TabTransformer (colunas como tokens). Embeddings são a linguagem comum para dados multimodais.